Dynamic Memory Management

Class 2 – malloc, VLAs, `defer`, and Arena Allocators

Jakub Piela

AGH University of Krakow · Faculty of Physics and Applied Computer Science
"KERNEL" CS Student Research Club · kernel.fis.agh.edu.pl

Academic Year 2025/2026

Outline

malloc: the safe idiom

Allocate on the heap; the size comes from the *object*, not the type:

```c
int* p = malloc(sizeof *p);          // one object
int* a = malloc(n * sizeof *a);      // array of n
if (!p || !a) { /* allocation failed -- handle it */ }
```

Recommendation

Write `sizeof *p`, not `sizeof(int)` — it stays correct if the type changes. **Never cast** the return of `malloc`: it is redundant in C and can hide a missing `<stdlib.h>`.[a]

[a] Source: J. Gustedt, *Modern C*, 3rd ed., Manning (2025), §13.1, Takeaway 13.1 #4.

calloc, realloc, free

```
int* a = calloc(n, sizeof *a);          // zeroed; n*size overflow-checked

int* tmp = realloc(a, m * sizeof *a);   // grow / shrink
if (tmp) a = tmp;                       // don't overwrite a on failure!

free(a);                                // free(NULL) is safe
a = nullptr;                            // defensive: avoid dangling pointer
```

Warning

`free`-ing twice or using memory after `free` is undefined behaviour. Assigning `nullptr` after `free` turns a silent use-after-free into an easy-to-spot null dereference.

The allocation family at a glance

Function	Behaviour
`malloc(size)`	Uninitialised; you compute `size` (can overflow silently)
`calloc(n, size)`	`n*size` bytes, zero-filled, overflow-checked
`realloc(ptr, size)`	Resize a block; may move it (copies contents)
`aligned_alloc(al, size)`	Block aligned to `al` (C11)
`free(ptr)`	Release a block; `free(NULL)` is a no-op

Tip

Heap allocation is *slow* relative to the stack and fragments over time. The rest of this lecture is about cheaper alternatives.

alloca: allocate on the stack

`alloca(n)` grabs `n` bytes from the current stack frame; the memory is released *automatically* when the function returns.

```c
#include <alloca.h>
void f(size_t n) {
    char* buf = alloca(n);   // no free() -- gone when f() returns
    /* ... use buf ... */
}
```

Warning

`alloca` is **non-standard** and has no failure signal — too big an `n` simply overflows the stack and crashes. Never call it in a loop (allocations accumulate until the function returns), and remember the lifetime is the whole *function*, not the enclosing block.

Variable-Length Arrays (VLAs)

A VLA is an array whose length is a run-time value — also on the stack:

```c
void f(size_t n) {
    int a[n] = {};              // VLA; C23 empty-init {} zeroes it
    // (pre-C23: leave uninitialised, then memset(a, 0, sizeof a))
}
```

Note

VLAs are optional since C11 (check `__STDC_NO_VLA__`) and carry the same stack-overflow risk as `alloca` for large or untrusted `n`. C23's empty initialiser `= {}` zero-initialises a VLA too (GCC/Clang, `-std=c2x`) — older C needed a manual `memset`.

A VLA as a cache: memoised Fibonacci

The parameter `cache[static n+1]` decays to a pointer; `static n+1` tells the compiler the array has *at least* n+1 elements:

```c
size_t fib(size_t n, size_t cache[static n + 1]) {
    if (n < 2)    return n;             // F0 = 0, F1 = 1
    if (cache[n]) return cache[n];      // already computed
    return cache[n] = fib(n - 1, cache) + fib(n - 2, cache);
}
size_t cache[n + 1];                    // caller: declare + zero
memset(cache, 0, sizeof cache);
printf("%zu\n", fib(n, cache));
```

From *Modern C*

Adapted from *Modern C*: recursion with an explicit cache whose precondition lives in the parameter type.[a]

[a] Source: J. Gustedt, *Modern C*, 3rd ed., Manning (2025), §7.3.

The problem: cleanup on every error path

Each early return must undo everything done so far — easy to get wrong:

```
int work(void) {
    char* a = malloc(N); if (!a) return -1;
    char* b = malloc(M); if (!b) { free(a); return -1; }   // easy to forget
    if (process(a, b) < 0) { free(b); free(a); return -1; } // and again...
    free(b); free(a);
    return 0;
}
```

The classic remedy is a single `goto cleanup;` label — workable, but noisy and easy to misorder.

Tip

For a fatal error in a short-lived program you can also just call `exit(EXIT_FAILURE)` — the OS reclaims all memory on process exit.

__attribute__((cleanup)) (GCC / Clang)

Attach a destructor to a variable; it runs when the variable leaves scope — even on an early `return`:

```
1 static void freep(void* p) { free(*(void **)p); }
2 #define CLEANUP(f) __attribute__((cleanup(f)))
3
4 void work(void) {
5     char* a CLEANUP(freep) = malloc(N);
6     char* b CLEANUP(freep) = malloc(M);
7     /* ... no manual free() anywhere ... */
8 }
```

Note

Cleanups fire in *reverse* order of declaration. This is the building block behind most portable `defer` macros.

Closures via GNU nested functions

A generic `defer` wants to capture local variables — a GNU *nested function* is a closure that can:

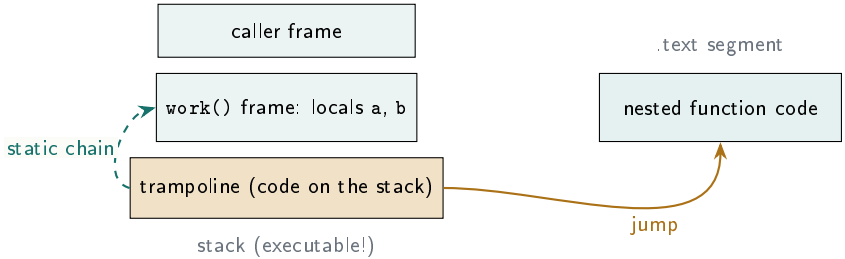
```
void work(void) {
    char* a = malloc(N);
    auto void cleanup(void) { free(a); }   // nested fn captures `a`
    /* register cleanup ... */
}
```

Warning

To make a *pointer* to a nested function, GCC writes a small **trampoline** onto the stack — which then must be executable. That breaks W^X and is a real security concern (see next slide).[a]

[a] Source: J. Gustedt, "Defer available in gcc and clang," `gustedt.wordpress.com`, 15 Feb. 2026.

How a trampoline works



The trampoline loads the *static chain* (a pointer to the enclosing frame's locals), then jumps to the function's code — giving the nested function access to `a` and `b`.

Note

This trampoline is only needed to take a *pointer* to a nested function. The standardised `defer` and `__attribute__((cleanup))` need *no* trampoline.[a]

[a] Source: J. Gustedt, "Defer available in gcc and clang," `gustedt.wordpress.com`, 15 Feb. 2026.

TS 25755: a real `defer` statement

A standardised `defer` runs a statement at scope exit, in reverse order — no trampolines, no manual labels:

```
1 void work(void) {
2     char* a = malloc(N);
3     defer free(a);            // runs last
4     char* b = malloc(M);
5     defer free(b);            // runs first
6     process(a, b);
7 }                             // both freed here, automatically
```

Note

Specified in **ISO/IEC TS 25755** and already available in recent GCC and Clang. Until it is everywhere, fall back to the cleanup attribute.[a]

[a] Source: J. Gustedt, "Defer available in gcc and clang," `gustedt.wordpress.com`, 15 Feb. 2026.

A portable `defer` fallback

Where TS 25755 is unavailable, a block-scoped macro built on the cleanup attribute covers most uses:

```c
static inline void run_defer(void (**f)(void)) { if (*f) (*f)(); }
#define defer  __attribute__((cleanup(run_defer))) void (*GENSYM)(void) =

void work(void) {
    char* a = malloc(N);
    defer ({ void _(void){ free(a); } &_; });   // GNU statement expr
}
```

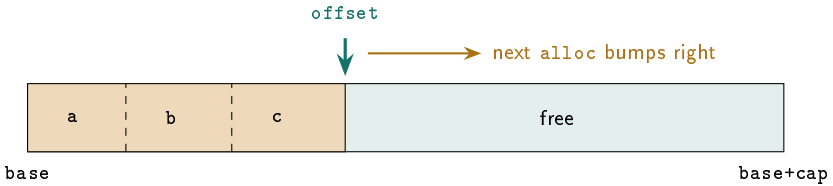
Tip

Gustedt's blog develops a fuller, more ergonomic macro. Prefer the native `defer` or the plain cleanup attribute when you can.[a]

[a] Source: J. Gustedt, "Defer available in gcc and clang," `gustedt.wordpress.com`, 15 Feb. 2026.

The idea: bump-pointer allocation

Reserve one big block; hand out slices by advancing an offset; free *everything at once*.



A minimal arena

```
typedef struct { char* base; size_t cap, off; } Arena;

void* arena_alloc(Arena *a, size_t n, size_t align) {
    size_t p = (a->off + (align - 1)) & ~(align - 1);  // round up
    if (p + n > a->cap) return nullptr;                // out of space
    a->off = p + n;
    return a->base + p;
}

void arena_reset(Arena *a) { a->off = 0; }             // free all -- O(1)
```

Note

Allocation is a pointer bump plus a bounds check; there is no per-object bookkeeping, so there is no per-object free.

Using an arena

```c
char buf[1 << 16];
Arena a = { buf, sizeof buf, 0 };

int* xs = arena_alloc(&a, 100 * sizeof *xs, alignof(int));
Node *n  = arena_alloc(&a, sizeof *n,        alignof(Node));
/* ... allocate freely, never free individually ... */

arena_reset(&a);   // reclaim everything at once
```

Tip

Back the arena with a stack buffer for short-lived scratch work, or with one big `malloc` for a longer-lived pool.

Trade-offs & strategies

Strengths

- Allocation $\approx$ one add + compare
- No fragmentation, no leaks
- Excellent cache locality
- Frees a whole phase in $O(1)$

Limitations

- No individual `free`
- Must size or grow the block
- You handle alignment
- Wrong lifetimes $\Rightarrow$ bugs

Common patterns: fixed scratch buffer · growable chunk list · per-frame / per-request arena · temporary "checkpoint" (save `off`, restore later).

Choosing an allocation strategy

Strategy	Use when	Avoid when
`malloc/free`	Independent lifetimes, large objects	Hot path with many tiny allocations
`alloca` / VLA	Small, bounded, function-local sizes	Loops or untrusted sizes
`defer` / cleanup	Many error paths to unwind	Hot loops (still pays the cleanup)
Arena	Many allocations, one shared lifetime	Individual frees needed

Key takeaway

Size from the object, never cast `malloc`, free on every path — or skip the problem with an arena.

References

- J. Gustedt, *Modern C*, 3rd ed., Manning, 2025. ISBN 978-1633437777 — §7.3 (recursion / cache), §13.1 (`malloc` and friends).
- J. Gustedt, "Defer available in gcc and clang," `gustedt.wordpress.com`, 15 Feb. 2026.
- ISO/IEC TS 25755 — *Defer mechanism for C*.